Lab 4: Exploratory Data Analysis (EDA)

Machine Learning for Bioinformatics

Item	Details
Course	Machine Learning (Bioinformatics)
Duration	3 hours (in lab) + submission by end of day
Prerequisites	Basic Python (variables, loops, functions, lists, dictionaries)
Tools	Python 3, Jupyter Notebook or Google Colab, pandas, numpy, matplotlib, seaborn, scikit-learn
Dataset	Breast Cancer Wisconsin (Diagnostic) – built into scikit-learn, no download needed
Work mode	Individual

1. Background

Before building any machine learning model, a data scientist must understand the data. Exploratory Data Analysis (EDA) is the process of inspecting, cleaning, summarizing and visualizing a dataset to discover its structure, quality problems, patterns and relationships. In bioinformatics, skipping EDA can lead to models that learn from errors, such as missing measurements, duplicated patient records or highly redundant features, rather than from real biology.

In this lab you will explore a clinical dataset of 569 breast tumor samples. Each sample is described by 30 numeric features computed from a digitized image of a fine needle aspirate (FNA) of a breast mass (for example radius, texture, perimeter and area of the cell nuclei). The target label is either malignant or benign.

2. Learning Objectives

After completing this lab, you will be able to:

- Load a biological dataset into a pandas DataFrame and describe its structure.
- Detect and handle missing values and duplicate records.
- Summarize features using descriptive statistics.
- Visualize distributions and relationships between features and a class label.
- Write short, evidence-based conclusions about which features may be useful for classification.

3. Setup (Provided Code)

Create a new notebook named Lab3_EDA_YourName.ipynb and run the code below in the first cell. It loads the dataset and deliberately introduces missing values and duplicate rows to simulate a messy real-world dataset. Do not change the random seed, so that everyone obtains the same results.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
```

```
# Load data
data = load_breast_cancer(as_frame=True)
df = data.frame.copy()
df['diagnosis'] = df['target'].map({0: 'malignant', 1: 'benign'})
df = df.drop(columns='target')

# Make the dataset 'messy' (do not modify)
rng = np.random.default_rng(42)
for col in ['mean radius', 'mean texture', 'mean area', 'worst perimeter']:
    idx = rng.choice(df.index, size=30, replace=False)
    df.loc[idx, col] = np.nan
df = pd.concat([df, df.sample(10, random_state=1)], ignore_index=True)
```

4. Task 1: Data Inspection and Cleaning

Goal: Understand the structure and quality of the dataset and produce a clean DataFrame called df_clean.

Requirements

1.1 Dataset overview

- Display the first 5 rows and the last 5 rows of df.
- Print the shape (number of samples and features) and the data type of every column.
- Count how many samples belong to each diagnosis class and report the percentage of each class. State in one sentence whether the classes are balanced.

1.2 Missing values

- Compute the number and percentage of missing values in every column. Show only the columns that have at least one missing value.
- Draw a bar chart of the missing-value counts per column (with title and axis labels).
- Fill the missing values of each affected column with the column median. In a markdown cell, explain why the median can be a better choice than the mean for biological measurements that contain outliers.
- Prove that no missing values remain (for example df_clean.isnull().sum().sum() should equal 0).

1.3 Duplicates

- Count the number of duplicated rows in df.
- Remove the duplicates and reset the index. Print the shape before and after removal.

1.4 Descriptive statistics

- Use describe() on df_clean and display the result.
- For the features mean radius, mean area and mean texture, report the mean, median, standard deviation, minimum and maximum in a small table that you create yourself.
- Write 2–3 sentences interpreting the table. For example, which feature has the largest scale, and why would this matter for distance-based algorithms such as k-NN?

5. Task 2: Visual Exploration and Insights

Goal: Use visualizations to discover which features separate malignant from benign tumors and which features are redundant.

Requirements

2.1 Univariate analysis

- Plot histograms (with KDE curve) for three features of your choice from the mean features group, arranged in one figure using subplots.
- Describe the shape of each distribution (symmetric, right-skewed, or left-skewed) in a markdown cell.
- Draw one boxplot of mean area for each diagnosis class and identify whether outliers are present.

2.2 Feature versus class

- For the same three features, plot histograms or violin plots separated by diagnosis (use the hue or x argument of seaborn).
- Compute the mean of each of the 10 mean features (columns beginning with "mean ") grouped by diagnosis, using groupby().
- Based on your plots and table, name the two features that you think best separate the two classes and justify your choice.

2.3 Correlation analysis

- Compute the correlation matrix of the 10 mean features and display it as a seaborn heatmap with numeric annotations.
- List all pairs of features that have a correlation greater than 0.9 (hint: use nested loops over the matrix; this is good practice for your Python skills).
- Create a scatter plot of mean radius versus mean area, colored by diagnosis.
- Write a conclusion (maximum 5 sentences) answering: Which features are redundant? What might happen if highly correlated features are all used in a model? Which preprocessing step would you perform before training a classifier?

6. Deliverables

- A single Jupyter notebook (.ipynb) containing all code, outputs and markdown explanations, executed from top to bottom without errors.
- Each task and sub-task must be labelled with a markdown heading (for example, "Task 1.2 Missing values").
- All figures must have a title, axis labels and readable legends.

8. Useful Functions (Hints)

Function	Purpose
df.head(), df.tail(), df.shape, df.dtypes, df.info()	Inspect structure
df.isnull().sum(), df.duplicated().sum()	Find missing values and duplicates
df.fillna(), df.drop_duplicates()	Clean data
df.describe(), df.groupby('diagnosis').mean()	Summary statistics

Function	Purpose
sns.histplot(), sns.boxplot(), sns.heatmap(), sns.scatterplot()	Visualization
df.corr(numeric_only=True)	Correlation matrix

Academic integrity: you may discuss ideas with classmates, but the code and written conclusions must be your own.